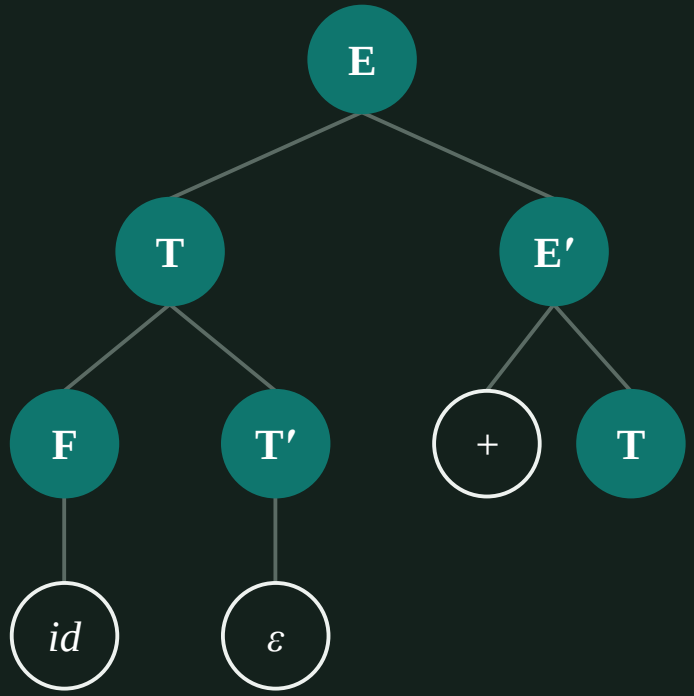
ParseViz

Interactive LL(1) Grammar Practice Studio

Hackathon project · Compiler Design
Unit II: Parsing (Top-Down) · PS-03

Team 09 · Section 9
B.Chetna (2420030098)
Vasavi (2420030785)
Aashna (2420030647)
Asleshitha (2420030740)

Submitted to: Uma Shankari



Problem and objectives

The problem

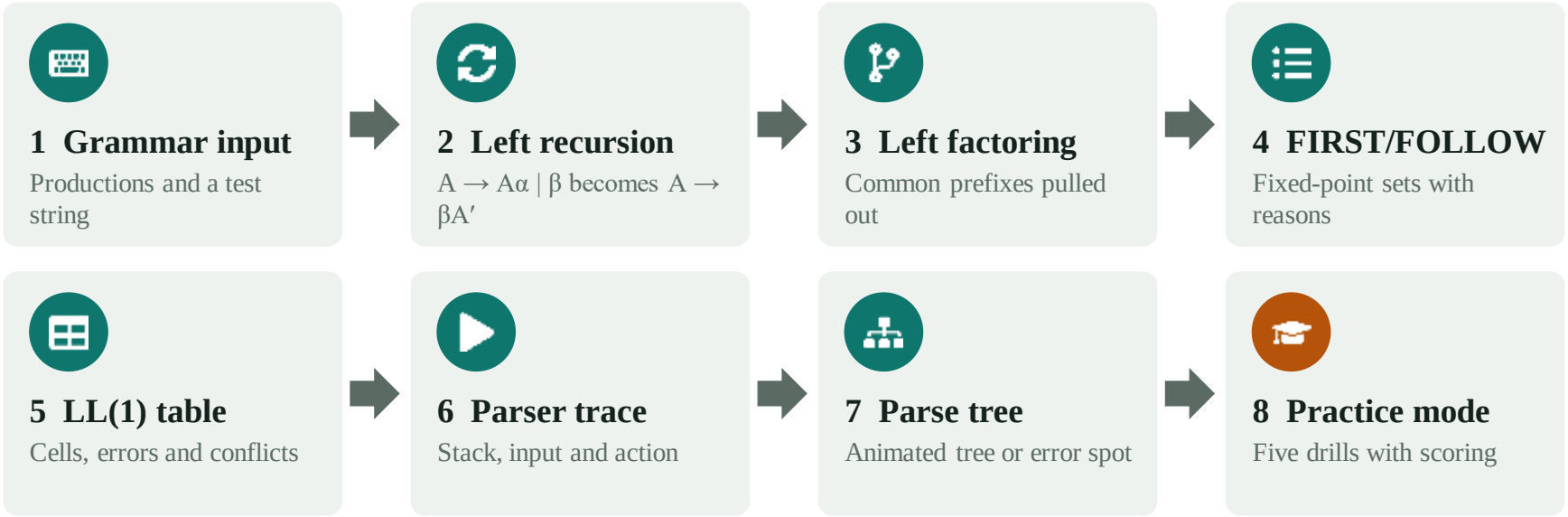
Grammars, derivations and parse trees are abstract. Students memorise the steps for removing left recursion and left factoring without knowing why they are needed.

Our solution

A studio that shows every step live, for any grammar, and lets students practise.

1. **Grammar and string input**
   valid, invalid or ε
2. **Left recursion removal**
   every step shown
3. **Left factoring**
   common prefixes
4. **FIRST and FOLLOW**
   with reasons
5. **LL(1) table**
   errors and conflicts
6. **Parsing trace**
   stack, input, action
7. **Parse tree or error**
   animated tree
8. **Practice mode**
   bonus challenge

System architecture



HTML + CSS interface | **JavaScript algorithms** | **SVG parse tree** | **Single file, runs offline**

Grammar transformation

Left recursion elimination

$$A \rightarrow A\alpha \mid \beta$$
$$\Rightarrow\ A \rightarrow \beta A' \quad A' \rightarrow \alpha A' \mid \varepsilon$$

Example

$E \rightarrow E + T \mid T$
$\Rightarrow E \rightarrow T\,E',\ E' \rightarrow +\,T\,E' \mid \varepsilon$

Why it is needed A top-down parser would expand E forever without reading input.

Left factoring

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$$
$$\Rightarrow\ A \rightarrow \alpha A' \quad A' \rightarrow \beta_1 \mid \beta_2$$

Example

$S \rightarrow a\,b\,c \mid a\,b\,d \mid a\,e \mid f$
$\Rightarrow S \rightarrow f \mid a\,S',\ S' \rightarrow e \mid b\,S'',\ S'' \rightarrow c \mid d$

Why it is needed One token of look-ahead cannot choose between alternatives with a common prefix.

FIRST, FOLLOW and the LL(1) table

Nonterminal	FIRST	FOLLOW
E	(id	$)
T	(id	+ $)
F	(id	* + $)
E′	+ ε	$)
T′	* ε	+ $)

FIRST and FOLLOW (expression grammar)

Every set member keeps the production that added it, so students see why, for example id ∈ FIRST(E) because E → T E′

Building M[A, t]

- Put A → α in M[A, t] for every t in FIRST(α).
- If α can derive ε, also for every t in FOLLOW(A).
- Two productions in one cell is a conflict: the grammar is not LL(1).

Production in the cell

Empty cell: syntax error

Conflict, e.g. dangling else

Parsing, parse tree and backtracking

Predictive parser

Stack, input and action at every step

Accepts id + id * id in 17 steps

Parse tree

Grows node by node with Play and step controls

Rejected input: red node and error position

Backtracking parser

Tries each alternative and undoes failures

34 steps, 6 backtracks for the same input

Stack (top first)	Remaining input	Action (first steps of the trace)
T E′ $	id + id * id $	Output E → T E′
F T′ E′ $	id + id * id $	Output T → F T′
id T′ E′ $	id + id * id $	Output F → id
id T′ E′ $	id + id * id $	Match "id"

Practice mode (bonus challenge)

1. **Accept or reject** predict the parser, then find the error token
2. **Remove left recursion** pick the correct rewrite
3. **Table lookup** which production is in M[A, t]
4. **FIRST / FOLLOW** select every member
5. **Next action** what does the parser do next

How it works

- 10-question rounds
- Easy, Medium and Hard levels
- Score, streak and bonus points
- Strings come from random derivations and token mutations
- The real parser decides every answer: no answer key

Testing, edge cases and how to run

Sample grammar	LL(1)?	Conflicts
Expression	Yes	0
Dangling else	No	1
ε productions	Yes	0
Left factoring	Yes	0
Indirect recursion	Yes	0
Parentheses	Yes	0

Edge cases handled

- ε productions
- Empty input string
- Indirect left recursion
- Several conflicts at once
- Unknown tokens

Run it

1. Clone or download from GitHub
2. Open index.html in Chrome or Edge
3. Pick a grammar, type a string

Conclusion: every step of top-down parsing is visible, checkable and practisable.

Output screens (1 of 2)



Output 1: Left recursion elimination
E and T are rewritten using E′ and T′, step by step



Output 2: Left factoring

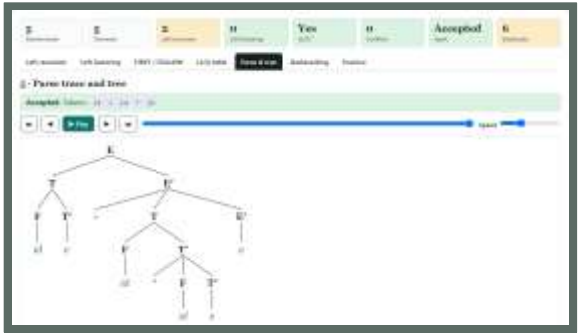


Output 3: FIRST and FOLLOW sets

Output screens (2 of 2)



Output 4: LL(1) parsing table, no conflicts



Output 5: Parse tree
with playback controls



Output 6: Practice mode
FIRST / FOLLOW drill with feedback